What 400 Million Pipeline Runs Reveal About Pipeline Failures

Constance Martineau · October 1, 2026



ASTRONOMER

Welcome to Astronomer Webinars

Upcoming Events

Virtual:

- **10/8** Migrating Control-M, AutoSys, Automic, and Tidal to Airflow
- **10/15** What every data engineer needs to know about ETL in 2026
- **10/29** Debugging Airflow's scariest issues

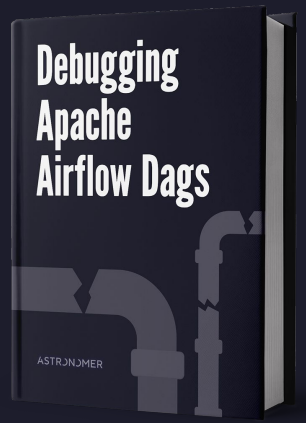
In-Person:

- **10/7** Airflow Birthday Party Seoul
- **10/7** Bangalore Airflow Meetup at Uber

Join the Airflow Slack!

s.apache.org/airflow-slack

Resource Highlight



A pipeline fails.
Someone asks what changed.

POSSIBLE CAUSES

- Schema drift
- Late arrival
- Environment change
- **Something you shipped**

You already know it (probably) wasn't you.

a column changed type

the file came late

an upstream team never read your DAG

POLL

What actually caused your last pipeline incident?

A	Code we shipped
B	An upstream schema or data change
C	Data arrived late, or not at all
D	Infrastructure, dependencies, or the environment
E	We never fully figured it out



Constance Martineau

STAFF PRODUCT MANAGER · ASTRONOMER

The data in this webinar comes from **Astro**, the managed Airflow platform I work on.

I used to write DAGs for a living. I sometimes cosplay as a data scientist.

linkedin.com/in/constance-martineau · github.com/cmarteepants

Three things to take out of this webinar

01

WHERE FAILURES LIVE

02

THE FIVE LAYERS

03

WHERE TO START

400 million production pipeline runs

Astronomer managed platform

2025 calendar year

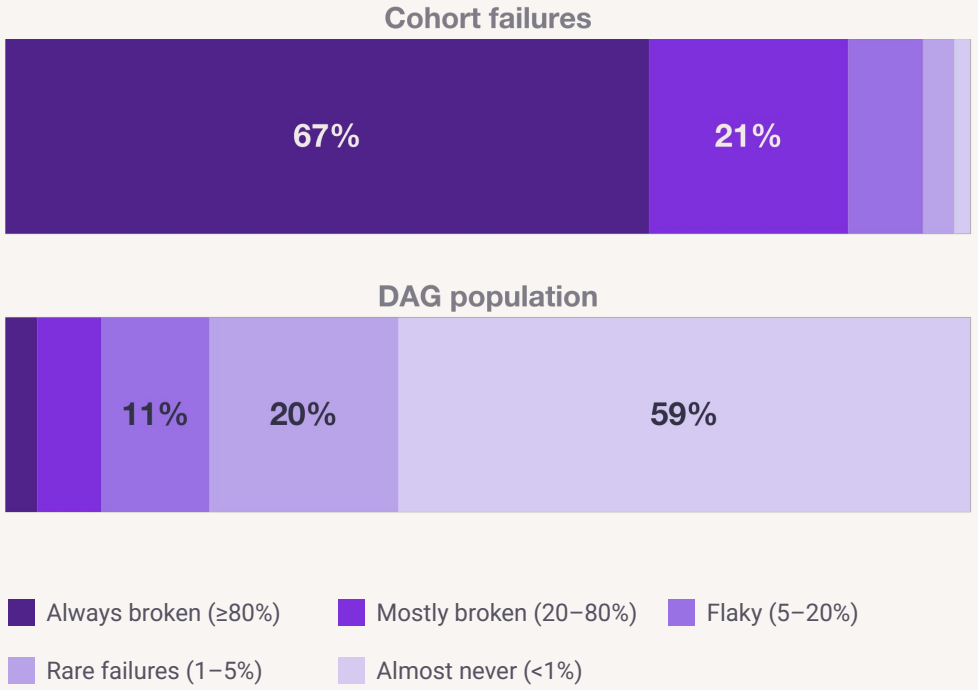
Paying customers only

3% of DAGs generate 67% of failures.

59% almost never fail.

Cohort average: about 7% (automated runs only).

Strip out the chronic failures and the rest runs at 2%.



Find your own 3%

include/find_your_3_percent.py

```python
def run_count(dag_id: str, state: str | None = None) -> int:
    params = {"run_type": "scheduled", "run_after_gte": SINCE, "limit": 1}
    if state:
        params["state"] = state
    return get(f"/dags/{dag_id}/dagRuns", **params)["total_entries"]

rows = []
for dag_id in all_dag_ids():
    runs = run_count(dag_id)
    if runs:
        rows.append((dag_id, runs, run_count(dag_id, "failed")))

rows.sort(key=lambda r: r[2], reverse=True)  # count, not rate
```

Where the failures live

Half

OF FAILURES

concentrated on a single runtime version

3–5×

FAILURE RATE

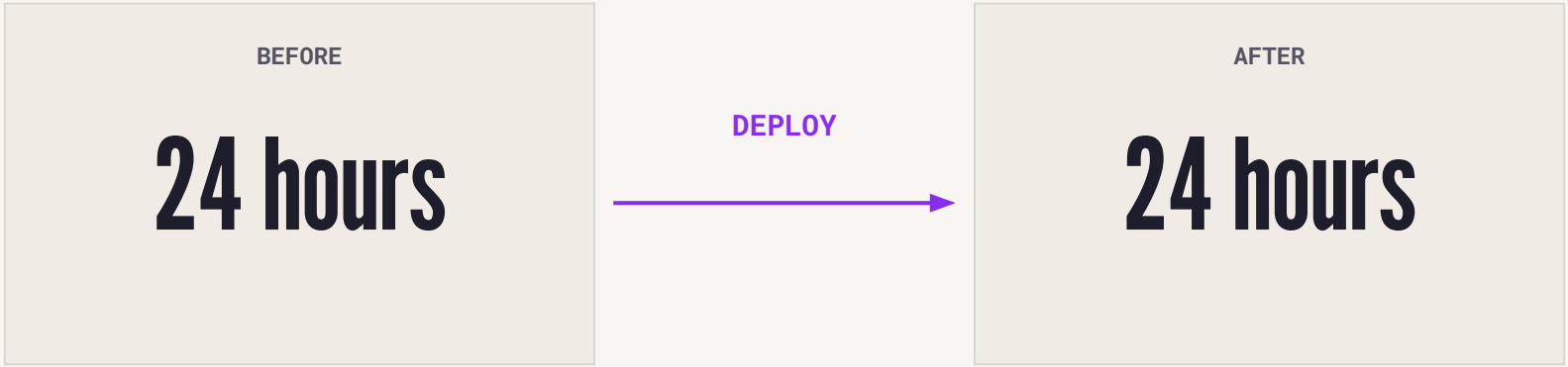
long-running vs. fast workloads

97%

FAILURE RATE

thousands of always-broken DAGs, all year

THE TEST

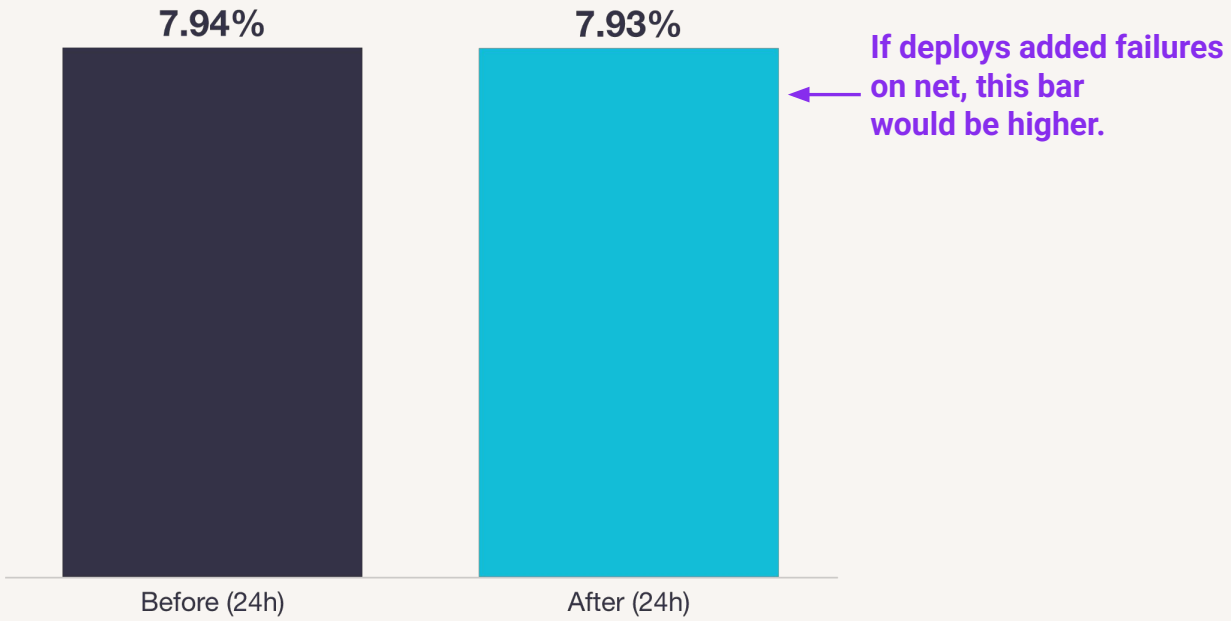


Same environment

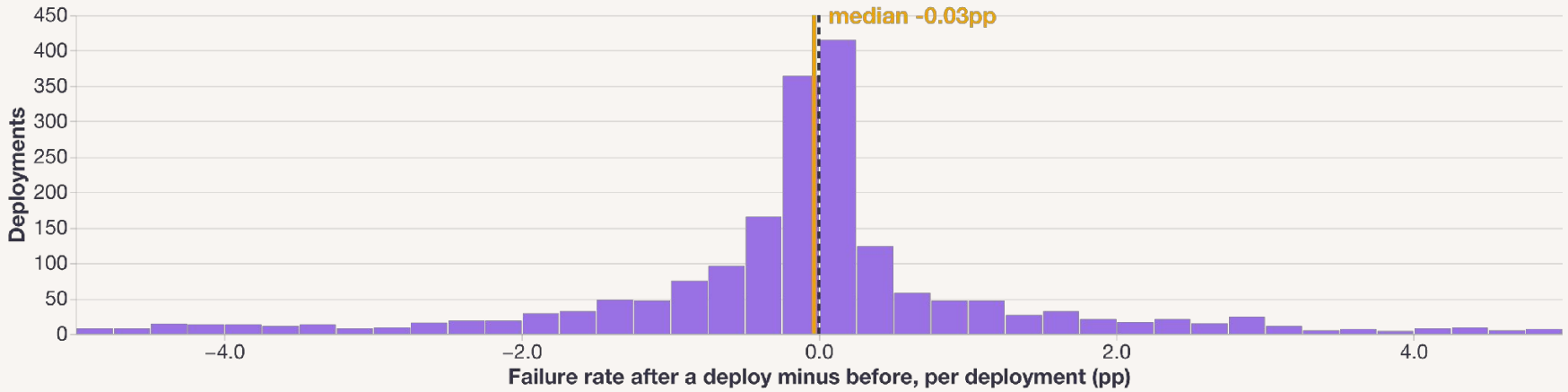
Same DAGs

Adjacent windows

7.94% before a deploy. 7.93% after.



Every environment, against its own before and after



2,148 Deployments · median -0.03pp · 58% got better or stayed flat

THE SURVIVORSHIP OBJECTION

DEV

15.1%

PROD

6.6%

CI/CD protects prod.

Limitations

WHAT I MEASURED

The deploy boundary is flat

Flat per environment, too

Dev fails more than prod

MY INFERENCE

Schema drift

Late arrivals

Shifting distributions

The deploy is an attention anchor

- 08:15 dag_pipeline_sales · failed
- 09:47 dag_reporting_daily · passed
- 11:03 dag_pipeline_sales · failed
- ▲ **14:32 Airflow 3.2 → 3.3**
- 14:33 dag_pipeline_sales · failed

“It worked fine before 3.3!”

A conversation you've had

DAG 1 · VENDOR API

`urllib3<2`

auth library pins it

⚡

DAG 2 · ML MODEL

`urllib3>=2`

model client requires it

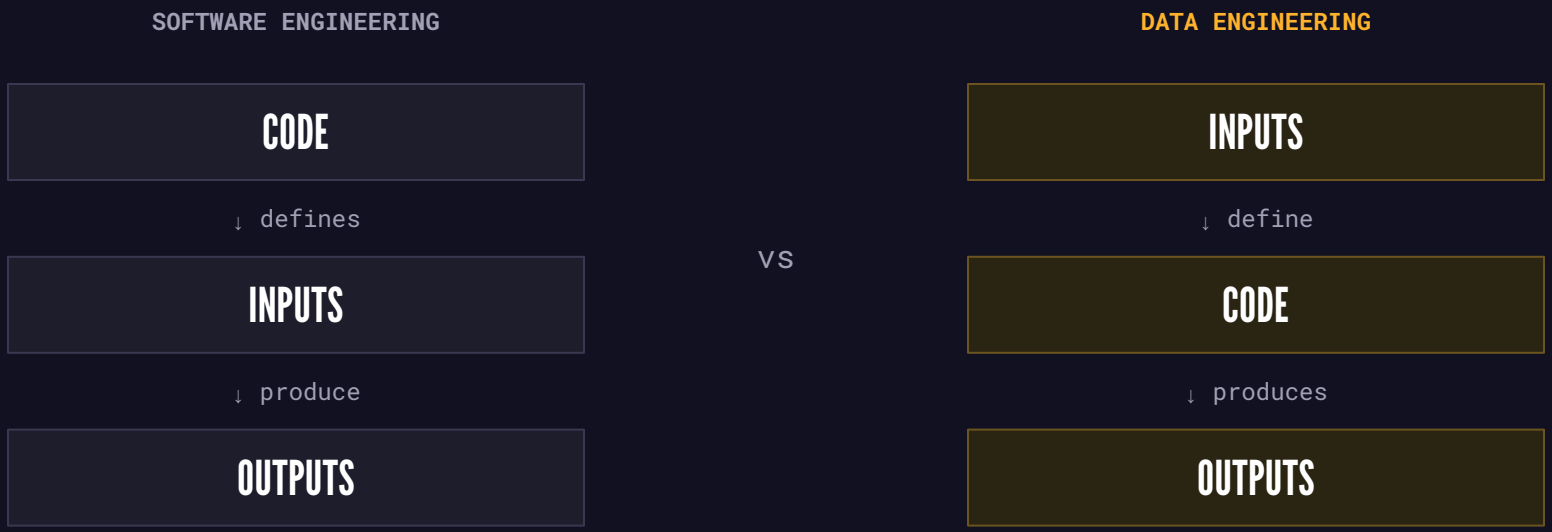
“Just pin urllib3 and downgrade the model client, or refactor one of the DAGs to run in its own pod.”

Senior platform engineer

Every check you built is green. And then upstream drops a column.

✓	code review	2 approvals
✓	pytest	247 tests passed
✓	DAG parse	0 import errors
✗	pipeline run	KeyError: 'user_type'

The mental model that doesn't transfer



Code drives outputs

Data drives your code

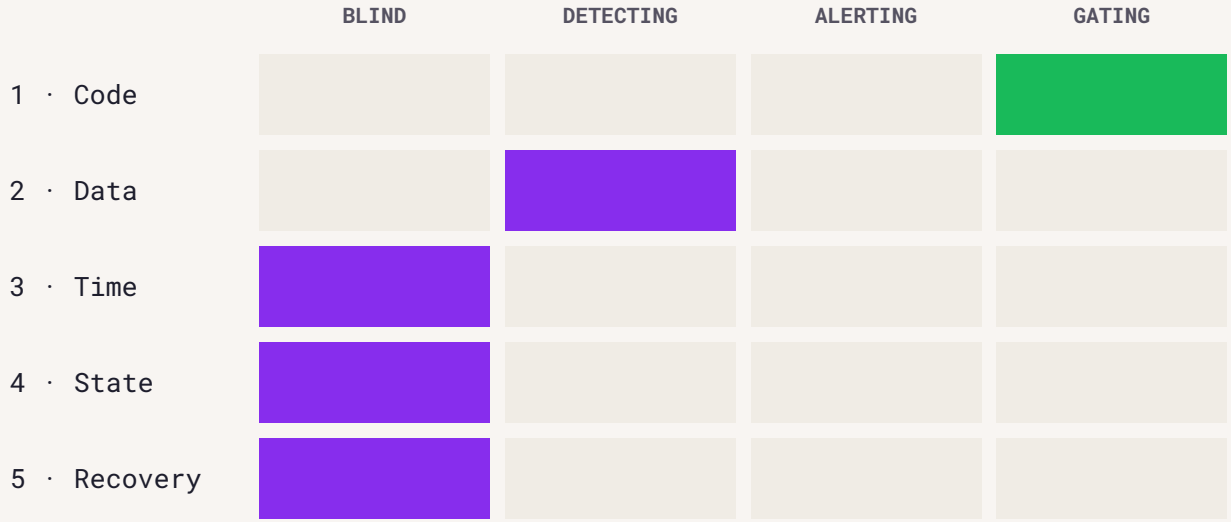
The five layers you actually operate

		WHAT YOU RUN	WHEN IT'S MISSING
1	CODE CORRECTNESS	Review, tests, lint, and the CI that runs them.	**The run fails on deploy. The one layer everybody has.**
2	DATA CORRECTNESS	Schema checks, null and uniqueness assertions, dbt tests, SQL check operators.	**The numbers are plausible, and wrong.**
3	TIME CORRECTNESS		
4	STATE SAFETY		
5	RECOVERY AND DEGRADATION		

The five layers you actually operate

		WHAT YOU RUN	WHEN IT'S MISSING
1	CODE CORRECTNESS		
2	DATA CORRECTNESS		
3	TIME CORRECTNESS	Freshness SLAs, arrival windows, completeness by a cutoff.	**The run succeeds, on stale data.**
4	STATE SAFETY	Write-audit-publish, staging tables, publish gates.	**Someone queries a half-written table.**
5	RECOVERY AND DEGRADATION	Quarantine paths, stale-but-labeled data, idempotent backfills.	**The same bad run repeats, on schedule.**

Every layer sits on the same four rungs



Your one reliable data quality check is **the CEO noticing.**

Increasingly, nobody does.

POLL

On data correctness, where does your platform sit?

A **Blind**: we find out from a human downstream

B **Detecting**: the signal exists somewhere

C **Alerting**: it reaches a person in time to act

D **Gating**: it stops the bad write without a human

Your orchestrator only fires when your code fails.

WHAT AN ORCHESTRATOR CAN SEE

The task raised. The run overran. The process died. Retries, callbacks, trigger rules, and deadline alerts all key off this, because execution is the only thing an orchestrator can observe.

WHAT IT CAN'T

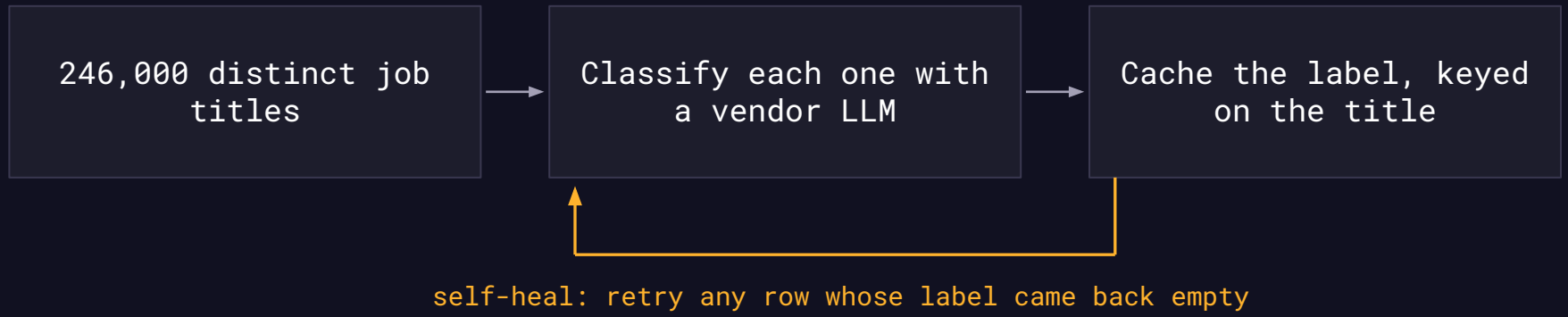
The column changed meaning. The file was late. The numbers are plausible and wrong. Vendors will sell you detection here. Gating needs a threshold, and a threshold is a decision about what's acceptable at your company.

Layer 3 is the gap

Data intervals | Assets | Sensors | Deadline alerts | 3.3: Deadlines page

Airflow 3.3 tells you which **runs** were late.
It still can't tell you which **data** was late.

The DAG


246,000 distinct job titles
Classify each one with a vendor LLM
Cache the label, keyed on the title
self-heal: retry any row whose label came back empty

The change

CHANGE 1 · THE KEY

```
- classify(title):"labels"[0]        "Data Engineer"
+ classify(title):"label"            NULL · no error · task succeeds
```

CHANGE 2 · THE FALLBACK

```
- COALESCE(label, 'other')           retry branch stays shut
+ removed                            the only thing holding it shut
```

246,000
ROWS PER PASS

Every run
WHOLE TABLE RE-SENT

4
DAYS UNDETECTED



0
EXCEPTIONS RAISED

0
FAILED TASKS

100%
OF THE OUTPUT WRONG

ASTRONOMER

THE ONE HARD RULE

Never automate recovery before you can tell good output from bad.

What they built afterward

4 **A MERGE GATE**

Fail the run if more than 10% of the batch came back unclassified, so a poisoned batch never reaches the cache.

5 **A CIRCUIT BREAKER**

Hard-fail before the LLM calls go out if the retry would re-select more than 5% of the cache. Halts, and waits for a human.

2 **STILL MISSING**

Nothing can tell a good label from a bad one. The taxonomy isn't fixed.

THE ONE HARD RULE, AMENDED

Never automate recovery before you can tell good output from bad...

...unless you can **bound the blast radius.**

After the fix, forty rows out of 246,000 were permanently stuck.

dsfh | qwe | ...

JOB TITLES, AS TYPED

FROM THE COMMIT THAT CLOSED IT

“prompt engineering can't fix it”

The containment: park anything that abstains twice, so it stops re-entering the retry pool.

40

ROWS · A ROUNDING ERROR

1,900

BILLABLE CALLS A DAY, INDEFINITELY

One thing here is new

	DATA PIPELINE	AI PIPELINE
THE INPUTS	MOVES	MOVES
THE FUNCTION	FIXED	MOVES

One moving part. Now two.

The same five layers, for an AI pipeline

		DATA PIPELINE	AI PIPELINE
1	CODE	Review, tests, CI	The same. Already solved.
2	DATA	Data quality checks, dbt tests	Eval suites, structured-output constraints
3	TIME	Freshness SLAs, arrival windows	Drift and model-version monitoring
4	STATE	Write-audit-publish, publish gates	Guardrails, output validators, human review
5	RECOVERY	Retries, quarantine, backfills	Fallbacks, quarantining a wrong answer

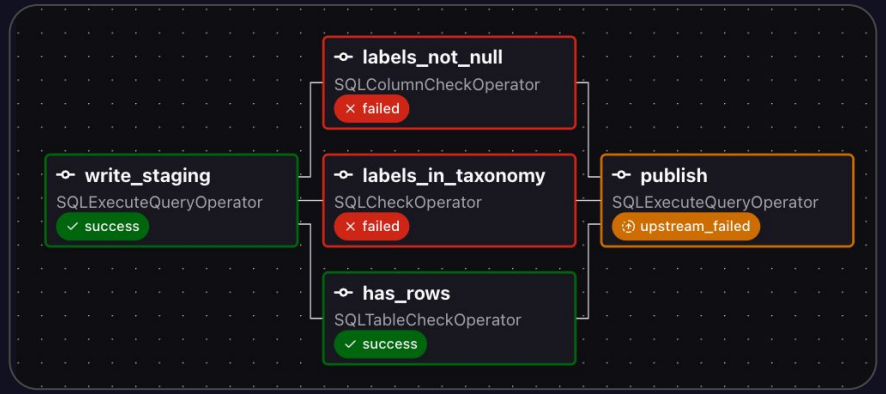
One of these transferred. **Four are being rebuilt from scratch.**

Layer 2: a check that gates

layer_2_data_gate_check.py

```python
labels_not_null = SQLColumnCheckOperator(
    task_id="labels_not_null",
    conn_id=CONN_ID,
    table=STAGING_PERSONAS_TABLE,
    column_mapping={"persona": {"null_check": {"equal_to": 0}}},
    retries=0,
)

labels_in_taxonomy = SQLCheckOperator(
    task_id="labels_in_taxonomy",
    conn_id=CONN_ID,
    sql=f"""
        SELECT CASE WHEN COUNT(*) = 0 THEN 1 ELSE 0 END
        FROM {STAGING_PERSONAS_TABLE} s
        LEFT JOIN {TAXONOMY_TABLE} t ON s.persona = t.label
        WHERE s.persona IS NOT NULL AND t.label IS NULL
    """,
    retries=0,
)
```



Layer 3: an arrival window

layer_3_arrival_window.py

```python
vendor_file_landed = SqlSensor(
    task_id="vendor_file_landed",
    conn_id=CONN_ID,
    sql=f"""SELECT COUNT(*) FROM {VENDOR_ORDERS_TABLE}
        WHERE load_date = CURRENT_DATE""",
    mode="reschedule",
    poke_interval=60,
    timeout=WINDOW.total_seconds(),
    retries=0,
)
```



Layer 4: a publish gate

layer_4_publish_gate.py

```python
@task(retries=0)
def audit(result):
    unclassified, total = (int(v or 0) for v in result[0])
    ratio = unclassified / total if total else 1.0
    if total == 0:
        raise AirflowFailException("Empty batch.")
    if unclassified > 25 and ratio > 0.10:
        raise AirflowFailException(
            f"Batch poisoned: {unclassified} unclassified rows."
        )
```



Layer 5: a circuit breaker

dags/layer5_circuit_breaker.py

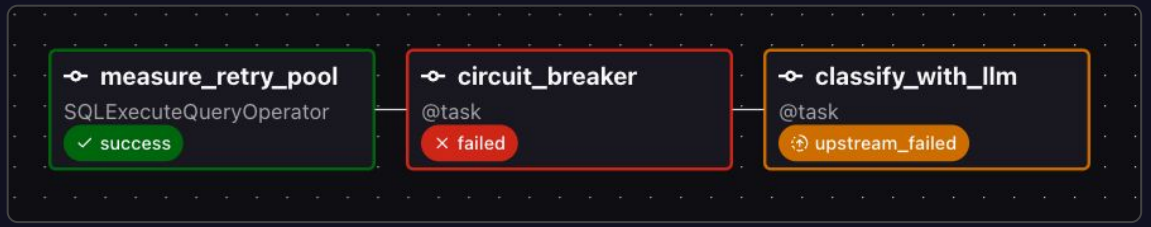
```
@task(retries=0)
def circuit_breaker(result):
    to_retry, cache_size = (int(v or 0) for v in result[0])
    ratio = to_retry / cache_size if cache_size else 0.0
    if to_retry >= 1000 and ratio > 0.05:
        raise AirflowFailException(
            f"Circuit open: retry would re-select {to_retry} rows. "
            "Halting before any model calls.")
    return to_retry

classify_with_llm(circuit_breaker(measure_retry_pool.output))
```

Layer 5: a circuit breaker

layer_5_circuit_breaker.py

```python
@task(retries=0)
def circuit_breaker(result):
    to_retry, cache_size = (int(v or 0) for v in result[0])
    ratio = to_retry / cache_size if cache_size else 0.0
    if to_retry >= 1000 and ratio > 0.05:
        raise AirflowFailException(
            f"Circuit open: {to_retry} rows to retry. No model calls."
        )
    return to_retry
```



Retry what's transient. Fail what isn't.

ai_ticket_router.py

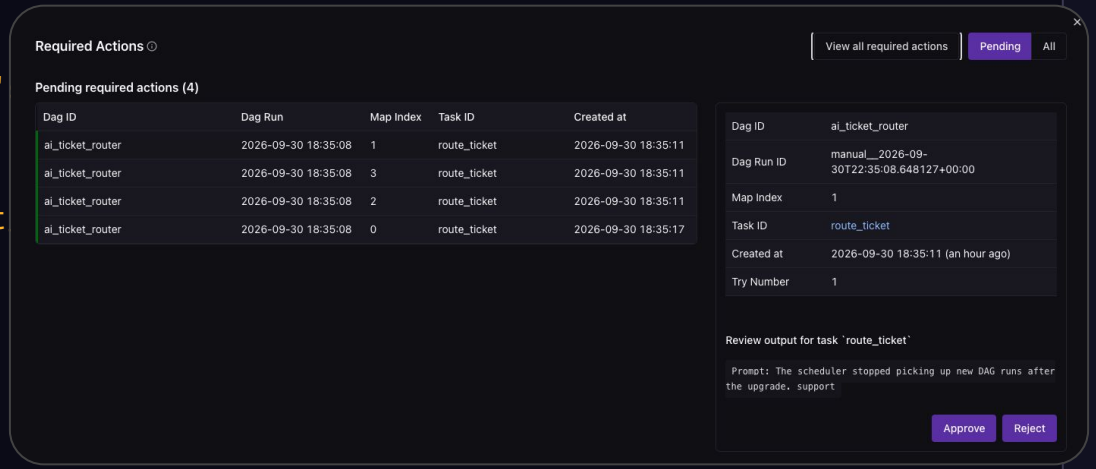
```
model_call_policy = ModelCallPolicy(rules=[
    RetryRule(exception="pydantic_ai.exceptions.ModelHTTPError",
              action=RetryAction.RETRY, retry_delay=timedelta(seconds=60)),
    RetryRule(exception=[ConnectionError, TimeoutError],
              action=RetryAction.RETRY, retry_delay=timedelta(seconds=10)),
    RetryRule(exception="pydantic_ai.exceptions.UnexpectedModelBehavior",
              action=RetryAction.FAIL),
    RetryRule(exception="pydantic_ai.exceptions.UsageLimitExceeded",
              action=RetryAction.FAIL),
])
```

And for an AI pipeline

dags/ai_ticket_router.py · common-ai provider

```
@task.llm(
    llm_conn_id="llm",
    output_type=Literal["billing", "
    model_id="typesafe:jev-1.13.0",
    require_approval=True,
    usage_limits=UsageLimits(request
    retry_policy=model_call_policy,
    retries=3,
)
def route_ticket(ticket: str):
    return ticket

route_ticket.expand(ticket=tickets)
```



And for an AI pipeline

dags/ai_ticket_router.py · common-ai provider

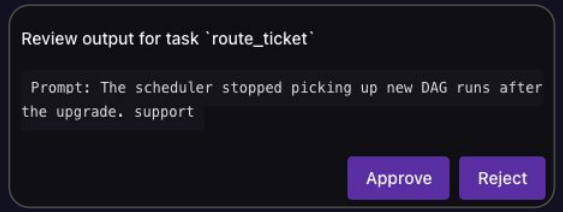
```
@task.llm(
    llm_conn_id="llm",
    output_type=Literal["billing", "support", "spam"],  # 2  a typed answer
    model_id="typesafe:jev-1.13.0",                     # 3  pinned version
    require_approval=True,                              # 4  a person signs off
    usage_limits=UsageLimits(request_limit=3),          # 5  bounded spend
    retry_policy=model_call_policy,                     # 5  retry what's transient
    retries=3,
)
def route_ticket(ticket: str):
    return ticket

route_ticket.expand(ticket=tickets)
```

And for an AI pipeline

ai_ticket_router.py

```python
@task.llm(
    llm_conn_id=LLM_CONN_ID,
    output_type=Literal["billing", "support", "spam"],  # 2  a typed answer
    model_id="typesafe:jev-1.13.0",  # 3  pinned version
    require_approval=REQUIRE_APPROVAL,  # 4  a person signs off
    usage_limits=UsageLimits(request_limit=3),  # 5  bounded spend
    retry_policy=model_call_policy,  # 5  retry what's transient
    retries=3,
)
def route_ticket(ticket: str):
    return ticket
```



Back to the runtime version

Half

the year's failures, on one runtime version

Upgrade on a cadence.

Rehearse it on a staging Deployment.

Use Otto to make code changes for you.

Long runs fail 3–5× more

Split it	`smaller tasks, each retryable on its own`
Defer the waiting	`deferrable operators free the worker`
Resume, don't restart	`task state store, Airflow 3.3`
Cap it	`execution_timeout`

Thousands of DAGs at 97%, all year

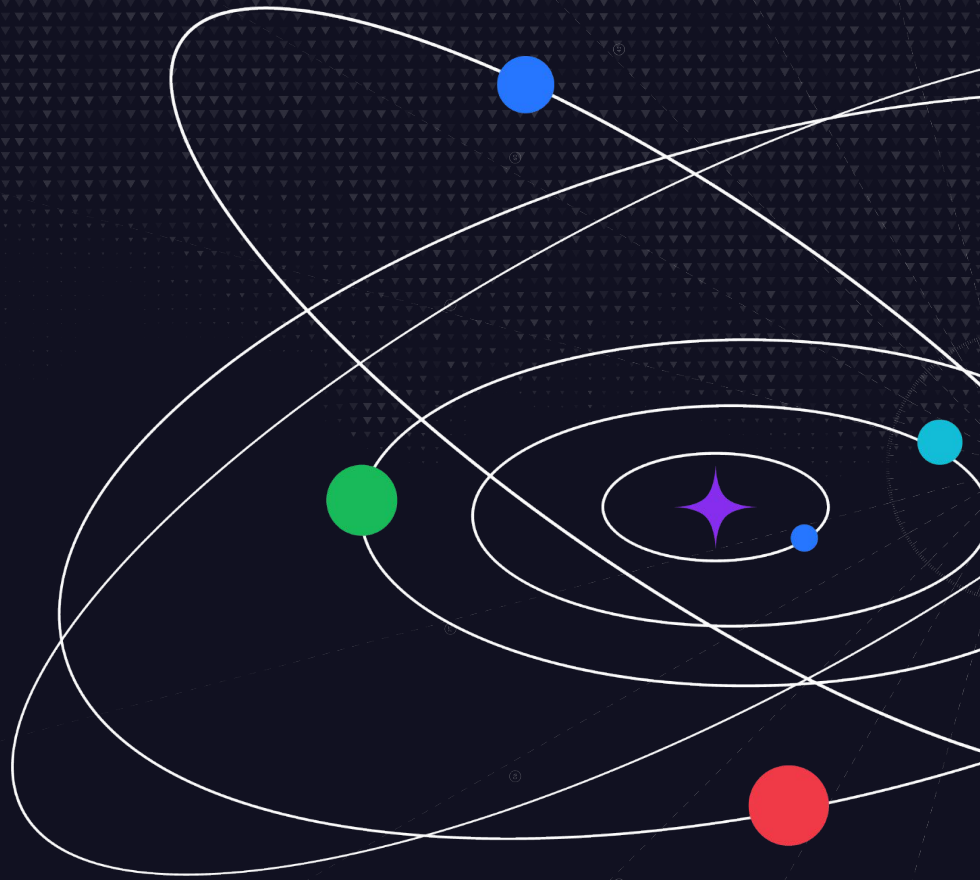
```
@dag(
    schedule="@hourly",
    max_consecutive_failed_dag_runs=5,   # pause after 5 in a row
)
def vendor_sync():
    ...

# or a default for every DAG, in airflow.cfg:
# [core] max_consecutive_failed_dag_runs_per_dag = 5
```

Where to start

1. Count your last ten postmortems
2. Find your 3%
3. Start with layer 3
4. Bound it, then gate it

You don't prevent failure. You bound it.



ASTRONOMER

Thank you. Questions?

astronomer.io/lp/signup



linkedin.com/in/constance-martineau